

Stack Data Structure

 <https://github.com/learn-co-curriculum/python-p3-dsa-stack-data-structure>  <https://github.com/learn-co-curriculum/python-p3-dsa-stack-data-structure/issues/new>

Learning Goals

- Define a **Stack** data structure.
- Identify common methods for a **Stack**.
- Identify common use cases for a **Stack**.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Stack** is a linear data structure that follows the principle of Last In First Out (LIFO)
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

In this lesson, we'll learn what **Stack** s are and what methods they commonly include. We'll discuss time complexity considerations when using **Stack** s and provide some common real-world examples of when **Stack** s are used. We'll also walk through an example algorithm, first coding it

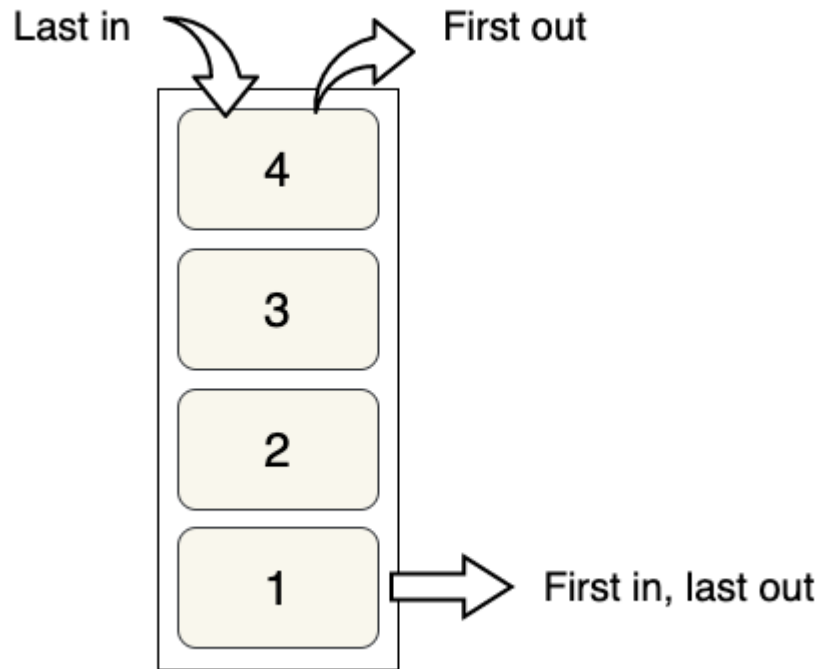
without using a **Stack** , and then with one.

Defining a Stack

A **Stack** is a linear data structure that allows you to store a list of data of some sort, and to add and remove values. Values in the stack are processed in **First In, Last Out** (FILO) order. In other words, the value that was added to the **Stack** most recently will be the first one removed. This can be contrasted with another similar data structure, a **Queue** , which is processed in **First In, First Out** (FIFO) order.

If we consider an airport security checkpoint as a real world example, the stack of bins is our **Stack** : when a passenger grabs a bin from the stack, it's the last bin that was added; in other words, **First In, Last Out**. (You can also think of it as **Last In, First Out**; the two terms are equivalent.) The line of passengers waiting to pass through security would be our **Queue** : the first person to join the line will be the first one through the checkpoint (**First In, First Out**).

It can be useful to think of a **Stack** as a vertical structure, like a stack of plates: we generally refer to adding items to, and removing them from, the *top* of the **Stack** :



Stack vs. list

You may be wondering why we wouldn't just use a list instead of implementing a **Stack**. After all, lists are also used to store a list of data, and also allow you to add and remove values. In fact, one way to implement a **Stack** (although not generally the best way) is by using a list as the underlying data structure — you'll be doing that in the next lesson.

Stacks have several benefits for certain problems when compared to lists. **Stack**s have a more limited set of methods for interacting with data compared to lists: with a **Stack**, you can only interact with the element at the *top*, whereas lists allow you to access and interact with elements at *any* position. This restriction is actually a good thing when it comes to solving certain kinds of problems, since it can guide you to a more elegant and easy-to-understand solution.

Stack Methods

The implementation of a **Stack** will vary depending on what's needed, but, at a minimum, generally includes the following methods:

- **push** : add an element to the top of the stack
- **pop** : remove the element at the top of the stack
- **peek** (or **top**) : return the value of the element at the top of the stack without removing it

In some implementations, you might also want to include a **limit** attribute, to indicate the maximum size of the **Stack** .

Fun Fact: the phrase **stack overflow** was originally coined to describe the situation of trying to push an item to a full **Stack** — it isn't just a place to find answers to coding questions! The reverse situation — trying to pop an item off of an empty **Stack** — is referred to as **stack underflow**.

Some other common methods you might see implemented include:

- **empty** / **full** : return true if the **Stack** is empty/full; false otherwise.
- **search(target)** : return the distance between the top of the stack and the target element if it's present; -1 otherwise.
- **size** : return the number of elements contained in the **Stack** .

Other methods are possible as well, of course: the methods the developer chooses to define in a given implementation of a **Stack** will depend on their particular needs.

Note also that there is nothing magical about the names of the methods. The names listed above are typically used by convention — and, as always, sticking to convention generally makes your code easier to read for other developers. But if you have a good reason for breaking convention in a particular circumstance, there's no reason you can't!

Time Complexity of Stack Methods

With the exception of **search** , all of the **Stack** methods listed above (for example, pushing an element onto the **Stack**) have time complexity of $O(1)$. The time complexity for **search** is $O(n)$.

Let's look at an example use of a **Stack** :

```
def reverse_string(string):  
    stack = []  
  
    for char in string:  
        stack.append(char)  
  
    reversed = ""  
    while stack:  
        reversed += stack.pop()  
  
    return reversed  
  
print(reverse_string("hello"))  
# => "olleh"
```

Here we are iterating through the string and adding each character to the **Stack**, which has a time complexity of $O(n)$. Then, we loop through the **Stack**, pop each character off and add it to the reversed string, again yielding a time complexity of $O(n)$. This gives $O(2n)$, which simplifies to $O(n)$.

When To Use a Stack

There are a number of practical use cases for a **Stack**. Some common ones include:

- The **call stack** [↗\(https://en.wikipedia.org/wiki/Call_stack\)](https://en.wikipedia.org/wiki/Call_stack) in computing
- Code compilers checking if brackets are balanced when a program is run
- Browser history and back/forward buttons
- Undo/redo in software programs

A **Stack** can also be used to help traverse a more complex data structure known as a **Tree**. (We'll learn about **Tree**s a bit later in this section.) For example, one common use of **Stack**s is in implementing a depth-first search through a **Tree**.

Example

Let's take a look at an example to see how we might use a **Stack** in solving a problem.

We want to write an **evaluate_keystrokes** method that will take as input a string that represents a series of keystrokes. The string may contain some number of occurrences of the **<** character, which indicates a backspace. We want our method to return the "interpreted" version of the string.

For example:

```
evaluate_keystrokes('abcde<fg<h')  
# => 'abcdfh'
```

```
evaluate_keystrokes('abcd<<<fg<h')  
# => 'afh'
```

```
evaluate_keystrokes('<fg<h')  
# => 'fh'
```

A solution that doesn't use a **Stack** might look something like this:

```
def evaluate_keystrokes(string):  
    i = len(string) - 1  
    result = ""  
    skip = 0  
  
    while i >= 0:  
        if string[i] == "<":  
            skip += 1  
            i -= 1  
        else:  
            result = string[i] + result  
            i -= 1
```

```

else:
    if skip > 0:
        i -= skip
        skip = 0
    else:
        result = string[i] + result
        i -= 1

return result

```

Here, we're iterating through our string from back to front and using a placeholder variable (`skip`) to keep track of how many times we need to backspace by skipping over characters. If we don't need to backspace, we simply add the current character to our `result` variable.

Now let's take a look at how we might approach this problem using a `Stack` :

```

def evaluate_keystrokes(string):
    stack = []
    for char in string:
        if char == "<":
            if len(stack) != 0:
                stack.pop()
        else:
            stack.append(char)

    result = ""
    while stack:
        result = stack.pop() + result

    return result

```

Every time we encounter the `<` , we "erase" the previous character by `pop` ping it off the stack, but only if the stack is not empty. We need to check for an empty stack to handle the special case of `<` being the first character in the string. By the end, all the characters that don't get "erased" remain in the `stack` , so we simply `pop` them off and add them to the `result` string.

This problem is one that lends itself pretty naturally to using a **Stack** , resulting in code that is simpler and easier to read.

Conclusion

In this lesson, we learned about the **Stack** data structure and the methods that an implementation of a **Stack** usually includes. We also talked about some real-world use cases for a **Stack** and went through an example algorithm. In the next lesson, you'll tackle implementing a **Stack** .

Resources

- [Wikipedia: Stack \(abstract data type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))  [https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))